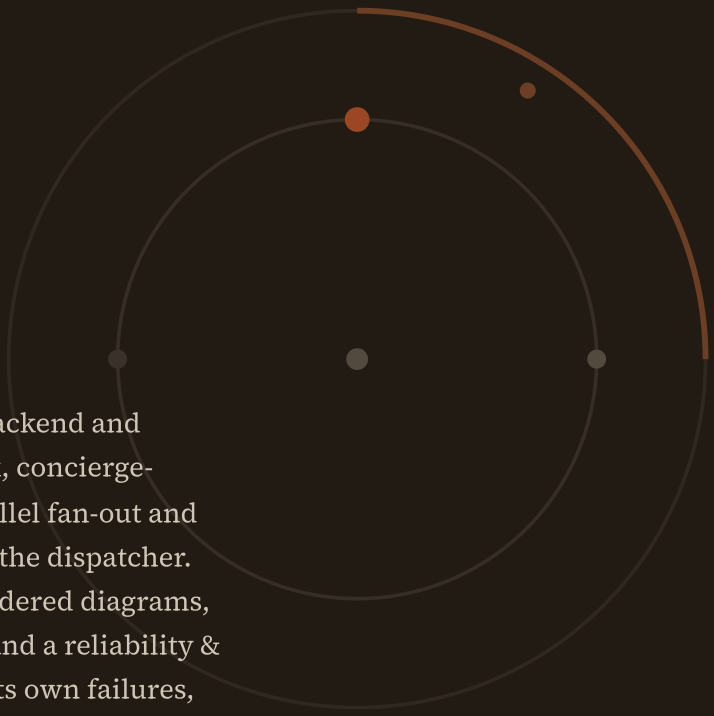
B E C K E T T

DESIGN PROPOSAL · OPS-151 → OPS-152 · --PROJECT BECKETT

Run of Show

a workflow engine the concierge conducts



A proposal to retire Plane as Beckett's execution backend and replace its one fixed ticket lifecycle with a per-task, concierge-authored workflow: stages, iteration budgets, parallel fan-out and gates composed at filing time — not compiled into the dispatcher. Revision 2 adds the engine's internal anatomy, rendered diagrams, three simulated deployments run stage-by-stage, and a reliability & observability design: how the system announces its own failures, and the explicit handshakes that guarantee a worker always knows where it is and what it is doing.

Author

Beckett — worker seat
Claude Fable 5 · high

Requested by

ro (owner)
Discord · OPS-151 · OPS-152

Date

12 July 2026
rev 2

Status

Proposal — no code
grounded in v3.2 source

IN ONE PAGE

TL;DR

Every piece of work Beckett takes on today — a copy tweak, a concurrency fix, a self-modification — marches through the same compiled-in pipeline: `todo → in_progress → in_review → done`, one implement seat, at most one review seat, three rework cycles, one live worker per ticket. That shape lives half in Plane (a remote SaaS ticket board we poll every few seconds) and half in a 3,200-line dispatcher switch statement. Changing it means changing Beckett's source and redeploying: adding *one* design stage cost us a new board, two new enum values, and surgery in five modules (OPS-111).

This document proposes making the workflow itself **data**: a small per-task DAG — the *run of show* — that the concierge writes when it files work. Stages, iteration caps, parallel fan-out, join rules and human gates become fields in a spec, interpreted by a generic engine that keeps everything already proven: worktrees and the task/branch tree, per-stage casting, review gates, the journal, the courier, steering. Plane is first demoted to a read-only mirror, then retired.

New in revision 2 (ro's review of rev 1): the engine's concrete anatomy — components, interfaces, the event log's exact vocabulary (§4); rendered architecture, lifecycle and fan-out diagrams throughout; three simulated deployments walked end-to-end with real specs, event logs and worker manifests (§7); and a reliability & observability design (§6) built around two requirements ro stated verbatim: *when something goes wrong we know about it*, and *the agent has clear handshakes so it always understands its environment*.

HOW TO READ THIS

Everything in §1 is fact, verified against the working tree with `file:line` receipts. Everything from §3 on is proposal. The two legacy lifecycles reappear in Appendix A as flow specs, which is the backward-compatibility argument in one page: if the engine can run *those two specs* identically, cutover changes nothing until we ask it to. Revision 2 is a strict superset of revision 1 — sections 4, 6 and 7 are new; carried sections are extended, not rewritten.

WHAT EXISTS · VERIFIED AGAINST SOURCE

1

Ground truth — how Plane runs Beckett today

Beckett's execution backend is a self-hosted **Plane** instance (`plane.0xbeckett.me`). The concierge files work as Plane issues; a poller diffs the board into events; a dispatcher turns events into worker processes in git worktrees. One paragraph of architecture, five subsystems of detail.

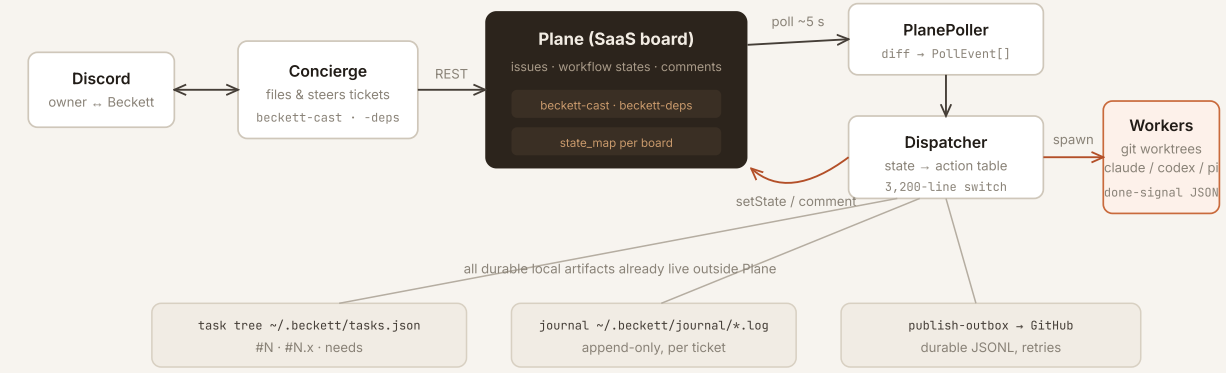


Fig. 1 — the v3 loop. The concierge writes to Plane; the poller diffs Plane into events; the dispatcher spawns workers and writes state back. All durable local artifacts (task tree, journal, outboxes) already live outside Plane.

1.1 The state machine and its enum

`TicketState` is a fixed eight-value union `src/plane/types.ts:26` — `backlog` , `todo` , `design` , `design_review` , `in_progress` , `in_review` , `done` , `cancelled` — with `done` / `cancelled` terminal `types.ts:39`. Each Plane board carries a `state_map` renaming these onto its columns `src/config.ts:129`. The dispatcher consumes poll events through one hardcoded dispatch table `src/dispatch/dispatcher.ts:1003-1062`:

EVENT / STATE ENTERED	GUARD	ACTION
`design`	INT board only, no live worker	spawn stage `design`
`in_progress`	no live worker	spawn stage `implement`
`in_review`	no publish hold, no live worker	spawn stage `review`
`done`	—	reap · drain orphaned steers · `promoteDependents()`
`todo` / `backlog` / `design_review`	—	park: abort+reap, commit WIP, keep worktree
`cancelled`	—	abort · reap · dispose
`comment_added`	live worker, author ≠ Beckett	`handle.nudge(body)` — steering

Stage names are string literals switched on in three places: the spawn table above, `castFor()` `dispatcher.ts:1998–2005` and the worker-done router `dispatcher.ts:2152–2161` (`design`, `design_check`, `implement`, `review`). A new stage is therefore a source change in at least three switch statements plus the enum, the poller's recovery list, and every board's `state_map`.

1.2 Casting, effort and the review gate

Casting is per-stage: a ticket description carries a fenced `beckett-cast` JSON block mapping stage name → `HarnessSpec` (`{harness, model, effort, reviewTier}`) `src/plane/cast.ts:29–59`. The `Casting` record is deliberately open-ended — arbitrary stage names already type-check `src/plane/types.ts:71–75` — but only the four hardcoded stage strings are ever spawned. Two consequential rules:

- **The review gate is inferred from a model knob.** `reviewTierFor()` `dispatcher.ts:2475–2479`: implement effort `low|medium` → *self* (one pass, straight to done); `high|xhigh|unset` → *fresh* (spawn an adversarial reviewer). An explicit `reviewTier` field overrides — a patch bolted on precisely because effort was overloaded.
- **Effort also sets the envelope** — advisory turn and wall-clock estimates via `ENVELOPE_BY_EFFORT` `src/dispatch/spawn.ts:217–222` (low 15 turns/10 min … xhigh 100 turns/60 min). These are supervision *estimates*, not hard kills — §1.6 covers what actually happens when a worker overruns or goes quiet.

Named cast presets live in `~/.beckett/presets.json`, hot-reloaded on every filing `src/plane/presets.ts:76–112` — a precedent this proposal leans on: per-task behaviour as user-editable data worked.

1.3 Iteration policy: four global constants

LOOP	CAP	ON EXHAUSTION
review fail → re-implement ("rework")	`MAX_REWORK_CYCLES = 3` `dispatcher.ts:199`	comment, park in `in_review` for a human
implement crash/timeout retry	`MAX_IMPLEMENT_RETRIES = 3` `dispatcher.ts:209`	publish WIP, move to `todo`
reviewer infra failure	`MAX_REVIEW_INFRA_RETRIES = 1` `dispatcher.ts:212`	park in `in_review`
design ↔ completeness check	`MAX_DESIGN_CYCLES = 2` `dispatcher.ts:202`	escalate to `design_review` with ⚠ note

These are compile-time constants. No ticket can ask for one shot, or six; the concierge's only lever over iteration is not filing the work.

1.4 One worker per ticket, worktrees, done-signals

The dispatcher enforces at most one live worker per ticket (the `workers` map, staffing dedup and FIFO pending queue `dispatcher.ts:450, 1313-1379`) under a global `max_workers` cap. A spawn allocates a persistent worktree at `<repo>/.beckett/worktrees/<id>` on branch `beckett/task-N[.x]`, installs the scope-guard and the done-signal schema, captures the review base SHA on first implement `dispatcher.ts:1573-1586`, and periodically checkpoint-commits live work (OPS-125) `dispatcher.ts:640-709`.

Workers finish by emitting a structured done-signal — delivered as the harness's native structured output, not a file or stdout convention `drivers/claude.ts:598`:

```
{ status: "complete" | "blocked" | "partial",   // the single edge the state machine branches on
  summary: string,
  filesChanged: string[],
  checksRun: string[] | null,
  blockedReason: string | null }              // spawn.ts:198-209
```

1.5 Steering, journal, courier, task tree

- **Steering.** A human comment on an in-flight ticket becomes a live nudge (receipts: delivered / queued / will-restart / dropped); with no live worker it is buffered in `pendingSteers`, persisted, and folded into the next worker's brief `dispatcher.ts:1070-1158, 1609-1635`. Plane comments are the transport; Discord is where the human actually typed.
- **Journal.** An append-only per-ticket log under `~/.beckett/journal/<ticket>.log`, fed fire-and-forget from worker events, read on demand via `beckett journal <ticket> --tail N` `src/progress/journal.ts`. Already fully local; already richer than Plane's comment trail.
- **Courier / publish.** Finishing a ticket publishes to GitHub via a durable JSONL outbox with 1 m/5 m/30 m retries; `dispatcher.courier(id)` hands exclusive publish ownership to a human `src/dispatch/publish-outbox.ts, dispatcher.ts:831-839`. Plane state writes themselves go through a second durable outbox (`advance-outbox`) because remote writes fail often enough to need one `src/dispatch/advance-outbox.ts`.
- **Task tree.** The user-facing `#N` / `#N.x` tree with `needs` dependencies lives in `~/.beckett/tasks.json` — a local, locked, versioned registry that already mirrors every Plane state into a branch status (`backlog→waiting, … in_review→review`) `src/task/store.ts:141-152`. Cross-ticket DAG promotion (`blockedBy` → promote on done) is dispatcher logic, not Plane's `dispatcher.ts:2889-2926`.

> **THE LOAD-BEARING OBSERVATION**
>
> Plane contributes exactly four things: issue storage, a state column per ticket, comments-as-transport, and a human-viewable board. Everything Beckett actually runs on — worktrees, casting, envelopes, done-signals, steering buffers, journal, outboxes, the `#N` tree, DAG promotion — is already local, already durable, and already ours. The migration in §9 is small precisely because of this.

1.6 How failure is noticed today — mostly, it isn't NEW IN REV 2

Rev 1 catalogued what the machine does; ro's review asked the sharper question — *how do we find out when it stops doing it?* The honest inventory, verified against source:

SIGNAL	WHAT ACTUALLY HAPPENS	WHO FINDS OUT, AND HOW
Worker overruns its effort envelope (e.g. the low tier's 600 s estimate)	**Nothing.** `ENVELOPE_BY_EFFORT` is advisory — "soft supervision estimates, never hard kills" `spawn.ts:208-222`. The worker runs on.	Nobody, until a human asks.
Worker goes silent (no progress events)	Driver emits a non-terminal `stalled` signal after `worker_stall_s` (default 300 s) `drivers/base.ts:367-385`; dispatcher escalates: strike 1 → status-check nudge, strike 2 → abort + retry `dispatcher.ts:1751-1786`.	The log file and the private journal. No channel message, no page.
Hard wall-clock backstop	`supervise.worker_hard_cap_s` (default 3600 s, floor 1800 s) `config.ts:269`; a 5 s watchdog kills the process tree and emits `finished/error_wall_clock_cap` `drivers/base.ts:352-409`.	The retry ladder sees it; a Plane comment may appear. Discord stays quiet.
Worker process dies without a done-signal	Driver synthesizes `finished/error_process_exit` with a null done-signal and the stderr tail `drivers/base.ts:327-343`; dispatcher commits WIP and retries up to `MAX_IMPLEMENT_RETRIES`.	Same: journal + logs. Silence from the operator's seat.
Ticket parked after cap exhaustion	A Plane comment via `postComment()` `dispatcher.ts:2810-2816`; the ticket sits in `in_review` / `todo`.	Only if a human happens to open the board.
Spend	Per-stage `SpendRecord` appended to a JSONL ledger (tokens, cost, duration, outcome) `src/spend.ts:7-31`.	On-demand reads. No budget alarm exists.

Two structural facts fall out. First, **detection exists but announcement does not**: every failure mode above does emit an event, but the events terminate in a log file, a private journal, or a Plane comment nobody is watching — the old user-facing Discord progress thread was deliberately removed `journal.ts:5-9`. From the operator's seat a dead worker and a thinking worker look identical: the practical experience behind "workers die silently at the 600 s cap." Second, **the worker's own picture of its environment is implicit**: its branch and worktree are communicated only as its `cwd`; its stage, its position in any larger plan, and its remaining budget are not stated anywhere it can read `spawn.ts:320-426`. It cannot check "am I in the right place?" because nothing tells it what the right place is. §6 designs both halves: announcement (§6.1–6.3) and self-orientation (§6.4).

WHY CHANGE · SIX CONCRETE FAILURES

2

The problem — one lifecycle, worn as a straitjacket

Nothing in §1 is broken code. The machine is careful, durable and observable-in-principle. What it cannot do is *vary*: the shape of work is compiled in, so every task — whatever its nature — is forced through the same pipe, and every new shape is a Beckett source change plus a deploy.

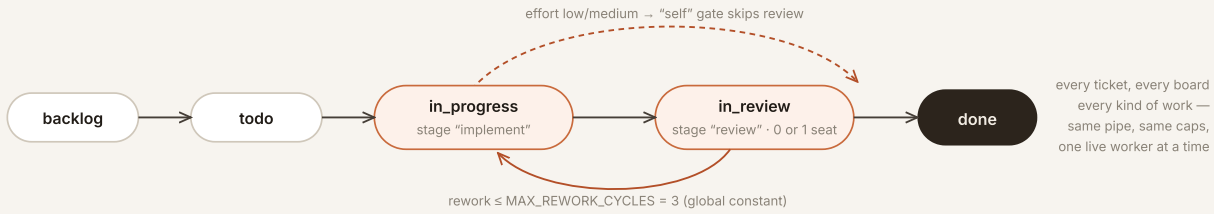


Fig. 2 — the compiled-in lifecycle. The only per-task levers are which models sit in each of two seats and an effort knob that doubles, implicitly, as the review-gate switch.

2.1 Case study: adding one stage took a board, an enum, and five modules

OPS-111 wanted exactly one new thing: a *design* stage with a human approval gate before implementation. Because the lifecycle is compiled in, the delivered change (OPS-111 → build) required: a **new Plane board** (INT) so OPS wouldn't grow columns; **two new enum values** (`design`, `design_review`) in `TicketState`; board-guarded arms in the dispatcher switch; a new `design_check` pseudo-stage with its own hardcoded cast (Haiku, low) and its own constant (`MAX_DESIGN_CYCLES`); poller `prime()` changes so restarts re-staff `design` but not `design_review`; and Plane-side state provisioning with the right column groups and colours `docs/design/int-flow.md · dispatcher.ts:2180–2239 · poll.ts:180`. That is the cost of *one* stage, designed by the book. The next shape — say, a research stage, or a second reviewer — costs the same again. The lifecycle has a marginal cost per variant that data should have made ~zero.

2.2 Iteration budgets are global, not per task

A README tweak and a subtle concurrency fix both get exactly three rework cycles, three crash retries, two design bounces (§1.3). The concierge — the seat with actual context about stakes, urgency and budget — cannot say "one pass, no retries, this is trivial" or "keep iterating up to six rounds against the eval, it's worth it." When a cap is wrong the options are: edit a constant in `dispatcher.ts` and redeploy, or babysit manually via state flips.

2.3 No parallelism inside a task

One live worker per ticket is structural `dispatcher.ts:450`. Things ro has wanted that this forbids outright:

- **Bake-offs** — three candidate implementations from different casts, one judge picks a winner. Today that is three hand-filed tickets, no shared base management, and the judging is ro reading diffs.
- **Panel review** — correctness, security and taste reviewers running *concurrently* on one diff. Today: one reviewer seat, serial by construction.
- **Research fan-out** — N scouts reading different subsystems before a synthesis stage. Today this only exists inside a single worker's subagents, invisible to the engine, unsteerable per-branch, and un-cast (the worker picks its own subagents; the concierge can't put Fable on one scout and Haiku on the rest).

The only parallelism the engine knows is between tickets via `beckett plan --needs` — the right tool for genuinely separate deliverables, wrong for branches of one task that share a worktree lineage and must join.

2.4 The workflow is inferred from a model knob

Whether a ticket gets reviewed at all is a side effect of effort (§1.2) — and an omitted effort silently buys the expensive fresh-review tier. `reviewTier` exists only to patch this overload. The shape of the pipeline should be a first-class, visible declaration, not something reverse-engineered from how hard the implement seat is thinking. (The presets doc already flags this as a foot-gun: "presets must always name an explicit effort" `docs/design/cast-presets.md §1`.)

2.5 State lives in someone else's database, structured data in a Markdown field

Every stage transition is a remote HTTP write to Plane — unreliable enough that a durable `advance-outbox` exists solely to retry them `src/dispatch/advance-outbox.ts` — and every reaction is gated on a ~5 s poll diff. Meanwhile the actually-structured task data (cast, dependencies, project, branch ref, start state, origin channel) is smuggled through Plane's description field as five fenced code blocks and an HTML comment `cast.ts:29-42`, because the board has no schema for any of it. The VID board bends video production onto the same six coarse states, and the client folds any unrecognised "started" column back to `in_progress` to survive human-added columns `client.ts:574`. These are the classic scars of a tool being used as a database it isn't.

2.6 The concierge can't compose; it can only pick

The concierge's authoring surface today is: title, body, criteria, board, entry state, and which models sit in the fixed seats (`--cast` / `--preset`). It cannot add a stage, split a stage, gate a stage, cap a loop, or fan anything out — the exact dimensions ro keeps asking to vary per task. Every one of those requests currently becomes an OPS ticket against Beckett's own source.

And — rev 2's addition to this list, from §1.6 — when any stage of that fixed pipeline dies, stalls or overruns, **the system knows and the operator doesn't.** A redesign of the workflow's shape that did not also fix who hears about failure would be redecorating a house with a broken smoke alarm.

THE REQUIREMENT, STATED ONCE

The concierge must be able to dictate, per task, how many stages there are, what runs in each, how many iterations each loop gets, and how much parallelism to spend — at filing time, as data, without touching Beckett's source or Plane's schema. And the engine that runs it must hold two reliability invariants: **every abnormal event is announced to a human surface within one minute**, and **every worker can prove to itself where it is and what it is doing.** Everything in §3–§8 is the smallest design we can find that delivers exactly that and nothing more.

THE PROPOSAL · WORKFLOW AS DATA

3

The proposal — a per-task flow spec

Replace the compiled-in lifecycle with a small, validated document — the **flow spec**, or *run of show* — attached to every task at filing time. The dispatcher's hardcoded state→action table becomes a generic interpreter over exactly four node kinds. Everything below the node boundary (worktrees, casting, envelopes, done-signals, journal, steering, courier) is reused unchanged.

3.1 Design principles

- **Small algebra, closed set.** Four node kinds: `worker`, `gate`, `fanout`, `join`. No expressions, no conditionals beyond pass/fail edges, no user-defined node types. Anything fancier must arrive as a fifth kind with its own design doc — that is the fence against the workflow-engine tarpit (§10.1).
- **Every loop is bounded by construction.** A spec that could run forever does not validate. Caps are per-node fields, defaulted, not global constants.
- **Today's behaviour is two specs.** The OPS and INT lifecycles must be expressible byte-for-byte (Appendix A), so cutover is a no-op until the concierge starts writing new shapes.
- **The concierge composes; the engine conducts.** Authoring is data + presets (§8); execution, recovery and budget enforcement are the engine's (§5). The worker never sees the flow — it still gets a brief, a worktree, and a done-signal schema.
- **The engine narrates itself.** NEW IN REV 2 Every transition the engine takes is an event some human surface hears about; every context a worker runs in is a contract the worker can read back. Reliability is not a subsystem bolted on in §6 — it is the run record (§5.1) doing double duty as the announcement stream.

3.2 The data model

```ts
// All of this validates with zod at filing time, like castings do today.
interface FlowSpec {
  version: 1;
  entry: NodeId;                    // where the run starts
  nodes: Record<NodeId, FlowNode>;
  budget?: {                        // hard ceilings for the whole run
    maxConcurrent?: number;         // this task's worker slots (≤ global max_workers)
    usd?: number;                   // spend cap across all seats
    wallClockS?: number;
  };
  supervise?: SuperviseSpec;        // rev 2: per-run alarm thresholds — §6.2, defaults apply
}

type FlowNode = WorkerNode | GateNode | FanoutNode | JoinNode;

interface WorkerNode {
  kind: "worker";                  // spawn one cast seat in the task worktree
  cast: HarnessSpec;                // existing {harness, model, effort} — unchanged
  brief?: string;                   // stage instruction appended to the ticket body
  artifact?: string;                // expected output path, e.g. "docs/design/<id>.md"
  onPass: NodeId | "done";         // edge taken on done-signal "complete"
  onFail: NodeId | "park";         // edge on "blocked"/"partial"/error; park = hand to human
  maxVisits?: number;               // re-entry cap for THIS node (default 3; rework generalised)
  retries?: number;                 // same-visit crash/timeout retries (default 3)
}

interface GateNode {                // a decision point: human or cheap model check
  kind: "gate";
  by: "human"                       // PARKED: no worker, zero tokens, concierge pings channel
    | { cast: HarnessSpec; rubric: string }; // LIVE: cheap check (generalises design_check)
  onPass: NodeId | "done";
  onFail: NodeId | "park";
  maxFails?: number;                // bounces before it parks anyway (design's ≤2, per node)
}

interface FanoutNode {              // N parallel branches of THIS task
  kind: "fanout";
  arms: { cast: HarnessSpec; brief?: string }[];  // explicit arms; or {template, n} sugar
  isolation: "worktree-each" | "shared";  // writers isolate; read-only scouts may share
  join: NodeId;                     // every arm's edges converge on one JoinNode
}

interface JoinNode {                // how parallel arms become one line again
  kind: "join";
  strategy: "all-merge"             // merge every arm's branch (deps-merge machinery reused)
    | "first"                       // first complete wins; engine aborts the rest
    | "quorum"                      // k-of-n verdict nodes agree (k in `quorumK`)
    | { judge: HarnessSpec };       // a cast seat reads all arms' diffs, picks/synthesises
  quorumK?: number;
  onPass: NodeId | "done";
  onFail: NodeId | "park";
}
```

The spec travels in a fenced `beckett-flow` block exactly where `beckett-cast` lives today (and, after migration, as a typed field in the local task record — §9). Casting stays a separate concern: the flow names seats; the cast fills them. A flow with no `beckett-flow` block is compiled from the cast exactly as today (effort → gate), so existing filings never break.

3.3 Validation — what the linter refuses at filing time

- Unknown edge targets; unreachable nodes; an `entry` that isn't a node; fanout arms whose edges escape their join.
- **Unbounded cycles:** every cycle in the graph must pass through a node whose `maxVisits` / `maxFails` is finite (all default finite; setting one to `Infinity` is simply not representable).
- **Budget sanity:** `maxConcurrent` ≤ global `max_workers`; a Fable seat anywhere in the spec triggers the existing confirm-before-cast handshake — flow authoring does not bypass the cast doctrine.
- Cast validation per seat via the existing `validateCasting` (blocked models, malformed efforts) `presets.ts:76-112`.
- **Supervision sanity** NEW IN REV 2 — a spec may relax alarm thresholds (§6.2) but not disable them: `heartbeatS` and `stallS` have floors, and `alarms: "off"` is, like `Infinity`, not representable.

3.4 What today's shapes look like (proof of containment)

The default reviewed lifecycle, as a spec — this is all of it:

```
{ "version": 1, "entry": "implement", "nodes": {
  "implement": { "kind":"worker", "cast":{"harness":"pi","effort":"high"},
                 "onPass":"review", "onFail":"park", "maxVisits":3, "retries":3 },
  "review":    { "kind":"gate",
                 "by": { "cast":{"harness":"claude","model":"claude-sonnet-5","effort":"high"},
                         "rubric":"criteria-vs-diff" },
                 "onPass":"done", "onFail":"implement", "maxFails":3 }
} }
```

One-pass work is the same spec minus the gate (`onPass:"done"`). The INT design flow adds two nodes in front — a design worker and a human gate — instead of two enum values, a board, and five modules. All three appear in full in Appendix A.

3.5 And what today cannot do at all

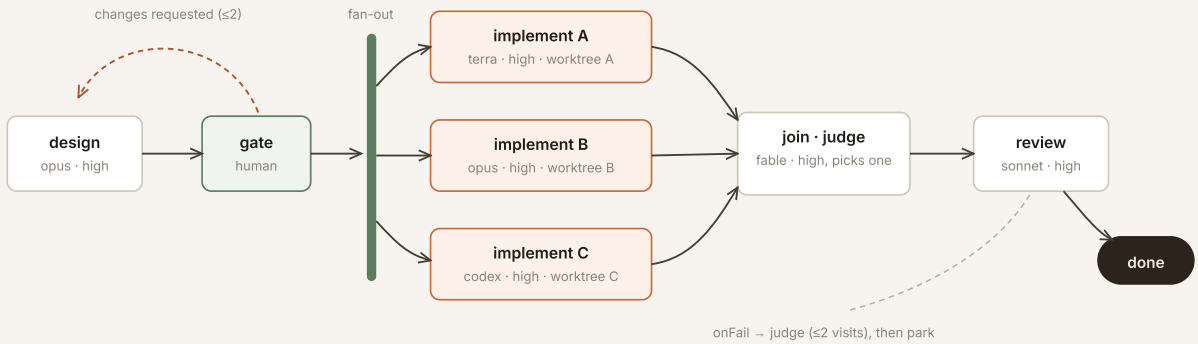


Fig. 3 — a bake-off run of show. Design → human gate → three isolated candidate implementations → a judge join that picks (or synthesises) a winner → fresh review → done. Six nodes of data. Under the current engine this is three hand-filed tickets, a human doing the judging, and no joint lineage.

Fig. 3 is six nodes of JSON. §7.2 walks its close cousin — a backend+frontend fan-out — through the engine event by event; §5.4 defines precisely how its arms fork and rejoin.

NEW IN REV 2 · COMPONENTS & CONTRACTS

4

Anatomy of the engine

Rev 1 named one component — "the stage manager" — and waved at the rest. This section pins the engine down to seven components with explicit interfaces, so the build can be sized honestly and reviewed piece by piece. Everything here lives in-process with the existing daemon; nothing is a service, a queue broker, or a new database. The boundaries are module boundaries, chosen so each piece is testable alone.

COMPONENT	ONE RESPONSIBILITY	EXISTS TODAY AS…
Flow Linter	Validate a `FlowSpec` at filing time: schema, graph shape, bounded cycles, budgets, cast doctrine (§3.3). Pure function; no I/O.	new; extends `validateCasting` `presets.ts:76`
Stage Manager	Per-run interpreter: fold the event log into a `FlowRun`, decide the next edge on every signal/alarm, enforce caps and budgets. The only component that understands node kinds.	replaces the state→action switch `dispatcher.ts:1003–1062`
Run Store	Append-only JSONL event log per run + a folded head snapshot. Single writer, fsync'd, crash-truncation tolerant. The source of truth.	the outbox idiom, promoted `advance-outbox.ts:49–104, spend.ts:31`
Seat Scheduler	Admission control: global `max_workers`, per-run `maxConcurrent`, FIFO queueing, dedup by `(run, node, visit, arm)`.	the staffing dedup + pending queue `dispatcher.ts:1313–1379`
Spawn Adapter	Turn a `SeatRequest` into a live worker: worktree, branch, scope-guard, brief, envelope, done-signal schema — plus, new, the stage manifest (§6.4). Returns a `WorkerHandle`.	today's spawn path, verbatim `dispatcher.ts:1479–1729 · spawn.ts:491`
Sentinel	Liveness: consume worker events into leases, raise typed alarms (stall, overrun, silent-exit, budget), drive the escalation ladder (§6.2). Never decides workflow — only reports.	generalises stall watch + hard-cap watchdog `base.ts:352–409`
Herald	Announcement routing: every run event is classified *trace / status / page* and delivered to the journal, the task's origin channel, or ro's DM (§6.3). Owns the "a human hears within one minute" invariant.	new; the gap §1.6 documents

Two supporting actors keep their existing names: the **Concierge** (authors specs, operates runs, answers pages) and the **Mirror** (Phase-1 write-only Plane projection, §9.2 — deleted in Phase 2).

4.1 The system, in one picture

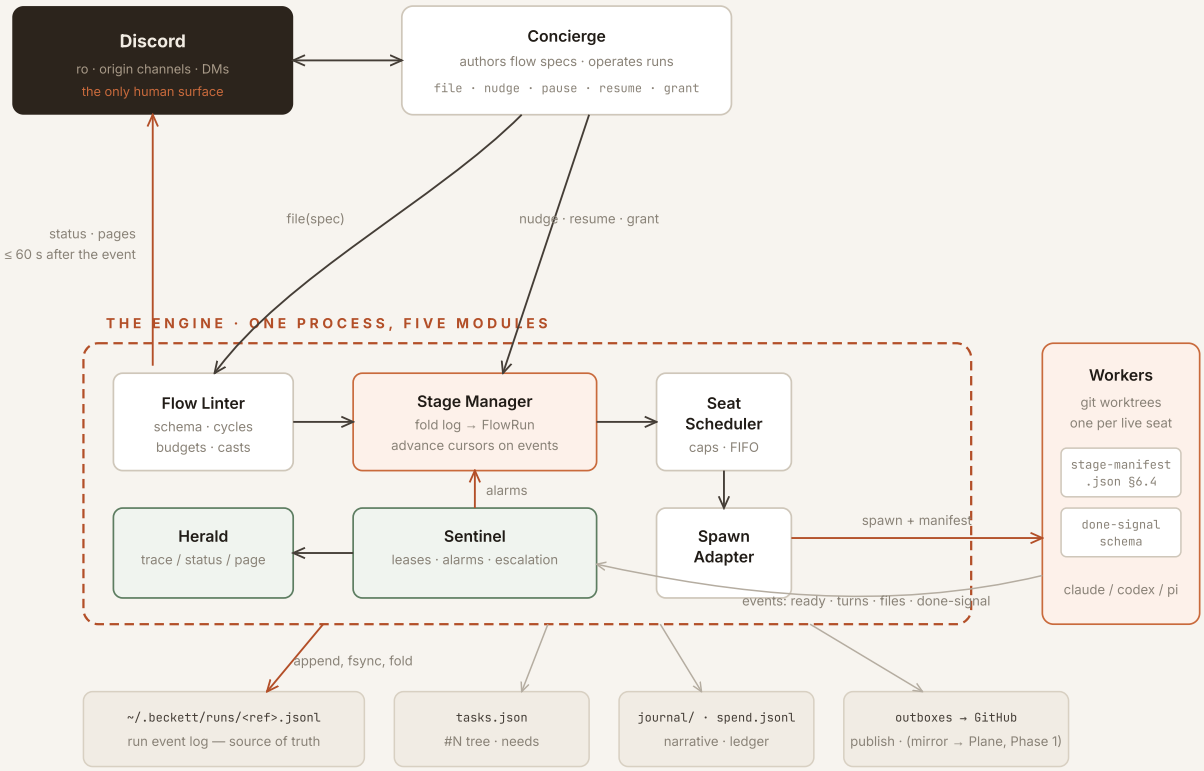


Fig. 4 — system architecture. The concierge authors and operates; the engine interprets, schedules, watches and announces; workers execute in worktrees against explicit contracts; all durable state is local JSONL. Discord is the single human surface — the Herald's ≤60 s announcement path (left edge) is the §6 invariant, drawn as an arrow because it is an interface, not an aspiration.

4.2 The three internal contracts

Three interfaces carry the whole design. If these signatures survive review, the components behind them can be built and tested independently.

```
// 1 · What the engine exposes to the concierge (and to `beckett` CLI verbs).
interface StageManager {
  open(ref: TaskRef, spec: FlowSpec): FlowRun;          // validate → run_opened → enter entry node
  onSignal(ref: TaskRef, node: NodeId, arm: number | null,
           signal: DoneSignal): void;                   // the ONLY workflow-advancing input
  onAlarm(ref: TaskRef, alarm: Alarm): void;            // Sentinel input; may retry, park, or ignore
  nudge(ref: TaskRef, text: string, node?: NodeId): Receipt;
  pause(ref): void; resume(ref, grant?: {extraVisits?: number}): void; cancel(ref): void;
}

// 2 · What the Stage Manager asks of the world when it wants a seat filled.
interface SeatRequest {
  ref: TaskRef; node: NodeId; visit: number; arm?: number;  // identity — also the dedup key
  cast: HarnessSpec;                                    // who sits down
  worktree: Path; branch: BranchRef; baseSha: Sha;      // where they sit
  briefParts: { body: string; criteria: string[]; nodeBrief?: string;
                priorArtifacts: ArtifactRef[]; steers: Steer[] };
  manifest: StageManifest;                              // §6.4 — the machine-readable handshake
  envelope: { turnCap: number; wallClockS: number };    // advisory, as today
}

// 3 · What a live worker looks like from inside the engine.
interface WorkerHandle {
  events: AsyncIterable<WorkerEvent>;   // session_started · worker_ready · turn_completed ·
                                        // file_change · checkpoint · stalled · finished
  nudge(text: string): Receipt;         // delivered / queued / will-restart / dropped — as today
  abort(reason: string): Promise<Sha>;  // commits WIP first; returns the checkpoint sha
  telemetry(): { turns: number; toolCalls: number; tokens: TokenUsage; wallClockS: number };
}
```

Nothing here invents transport: `events` is the driver event stream that already exists `types.ts:143, base.ts:89-92`, `abort` is today's commit-WIP-then-kill path `dispatcher.ts:2242-2259`, and `onSignal` is the done-signal router with its switch deleted. The one genuinely new input is `onAlarm` — the Sentinel's channel — and the one new field is `manifest`.

4.3 How an advance happens — the exact order of operations

1. **Append** the triggering event (`signal_received` , `alarm_raised` , an operator verb) to `runs/<ref>.jsonl` ; fsync. If this write fails, nothing happened.
2. **Fold** the log into the `FlowRun` head (§5.1). Folding is pure and deterministic — same log, same state, every time.
3. **Decide**: the Stage Manager computes the edge set the new state enables (pass edge, fail edge, retry, join-completion check, cap-exhaustion park, budget park).
4. **Gate**: each intended spawn is checked against budgets and caps *before* a seat is requested; a refusal becomes `budget_hit` + park, not a queued surprise.
5. **Act**: seats are requested from the Scheduler keyed by `(ref, node, visit, arm)` — the key makes replay idempotent: a crash between append and spawn re-derives the same request and dedups against any seat that already started.
6. **Announce**: the Herald classifies the appended events and delivers (§6.3). Announcement is after the fsync, so a page never refers to state that didn't commit.

4.4 The run-event vocabulary — closed, like the node algebra

Every durable fact about a run is one of these nineteen events. The fold (§5.1), the Herald's routing (§6.3), `beckett status`, and the simulations in §7 all read from this single vocabulary — there is no second, informal channel.

EVENT	EMITTED WHEN	KEY PAYLOAD
`run_opened`	spec validated, run created	frozen spec, origin channel
`node_entered`	a cursor arrives at a node (visit N)	node, visit, arm?
`seat_spawned`	Spawn Adapter launched a worker	seat key, cast, worktree, branch, baseSha
`worker_ready`	worker completed the §6.4 readiness handshake	manifest hash, observed branch/sha
`worker_refused`	worker's environment check failed — it declined to start	expected vs observed
`progress_noted`	lease renewal rollup — at most one per minute per seat	turns, files touched, tokens
`checkpoint_committed`	periodic WIP checkpoint (OPS-125 cadence)	sha
`signal_received`	done-signal arrived (or was synthesized on crash)	status, summary, subtype
`edge_taken`	Stage Manager moved a cursor	from, to, why (pass/fail/retry)
`gate_decided`	a gate resolved	by (human/model), verdict, rubric score
`arm_joined`	an arm reached its join barrier	arm, branch, status
`join_resolved`	join strategy concluded	strategy, winner/merge order/votes
`alarm_raised` / `alarm_cleared`	Sentinel verdicts (§6.2)	type, seat key, evidence
`nudge_delivered`	steering reached a seat (or its buffer)	receipt, target
`parked` / `resumed`	run handed to / taken back from a human	node, reason; grant used
`budget_hit`	a pre-spawn gate refused (§4.3 step 4)	ceiling, spend so far
`run_done` / `run_cancelled`	terminal	outcome, totals (spend, seats, wall-clock)

4.5 Storage layout — four files, one idiom

```
~/.beckett/
  runs/<ref>.jsonl        // the run event log — append-only, fsync'd, source of truth
  runs/<ref>.head.json    // folded FlowRun snapshot — cache only, rebuilt from the log at will
  tasks.json              // #N tree — unchanged (store.ts); run status projected into branch status
  journal/<ticket>.log    // human-narrative journal — unchanged, gains node context per line
  spend.jsonl             // SpendRecord ledger — unchanged (spend.ts), now keyed by seat, not stage
```

One idiom throughout: append-only JSONL with a tolerant reader — the exact mechanics the codebase already trusts in the publish-outbox, the advance-outbox and the spend ledger `publish-outbox.ts · advance-outbox.ts:49–104 · spend.ts:46`. No SQLite, no daemon-owned lock server; the Run Store is a module, not a service. If a head snapshot is ever corrupt or stale it is discarded and re-folded — corruption of the *log* is the only fatal case, and it fails loudly at the exact line, like the outboxes do today.

RUNTIME · THE STAGE MANAGER

5

Execution semantics

The engine — call it the **stage manager** — is a per-task interpreter with one job: hold a durable `FlowRun` record, advance cursors along edges when done-signals arrive, and enforce budgets. It replaces the dispatcher's state-switch; it deliberately does not replace the spawn path, the drivers, or any durability machinery.

5.1 The run record

```
interface FlowRun {
  taskRef: string;                // "#42.1" — the existing tree ref stays the primary key
  spec: FlowSpec;                 // frozen copy at filing time (edits create a new rev)
  status: "running" | "parked" | "done" | "cancelled";
  cursors: Cursor[];              // exactly 1, except between fanout and join
  visits: Record<NodeId, number>; // per-node re-entry counters (replaces reworkCount et al.)
  parked?: { node: NodeId; reason: string; since: string };
  spend: { usd: number; byNode: Record<NodeId, number> };
  leases: Record<SeatKey, { renewedAt: string; state: "live"|"quiet"|"expired" }>;  // rev 2 — §6.1
}
interface Cursor { node: NodeId; arm?: number; workerId?: string; branch?: string; }
```

Runs are event-sourced: every transition (`node_entered`, `signal_received`, `edge_taken`, `parked`, `budget_hit`… — the full vocabulary is the §4.4 table) is appended to `~/.beckett/runs/<ref>.jsonl` and the record above is just the fold. This is the same durability idiom the codebase already trusts — the journal, the advance-outbox and the publish-outbox are all append-only JSONL `journal.ts · advance-outbox.ts:49-104` — promoted to being the source of truth instead of a shadow of Plane's.

5.2 How a stage runs — nothing new below the node

Entering a `worker` node calls today's spawn path verbatim `dispatcher.ts:1479-1729 · spawn.ts:491`: allocate/reuse the task worktree, install scope-guard + done-signal schema — plus, rev 2, the stage manifest (§6.4) — build the brief (ticket body + node brief + drained steers), apply the effort envelope, register checkpoint commits. The node completes when the done-signal arrives: `complete` takes `onPass`; `blocked` / `partial` / process-error takes `onFail` (after same-visit retries for infra failures, preserving today's crash/auth/rate-limit ladder `dispatcher.ts:1844-1930, 2356-2442`). The LIVE/PARKED distinction survives intact: `worker` and model-`gate` nodes are LIVE; human gates and park are PARKED — no process, zero tokens, restart-inert, exactly the property `design_review` proved out `poll.ts:242-293`. Fig. 5 is this paragraph, drawn.

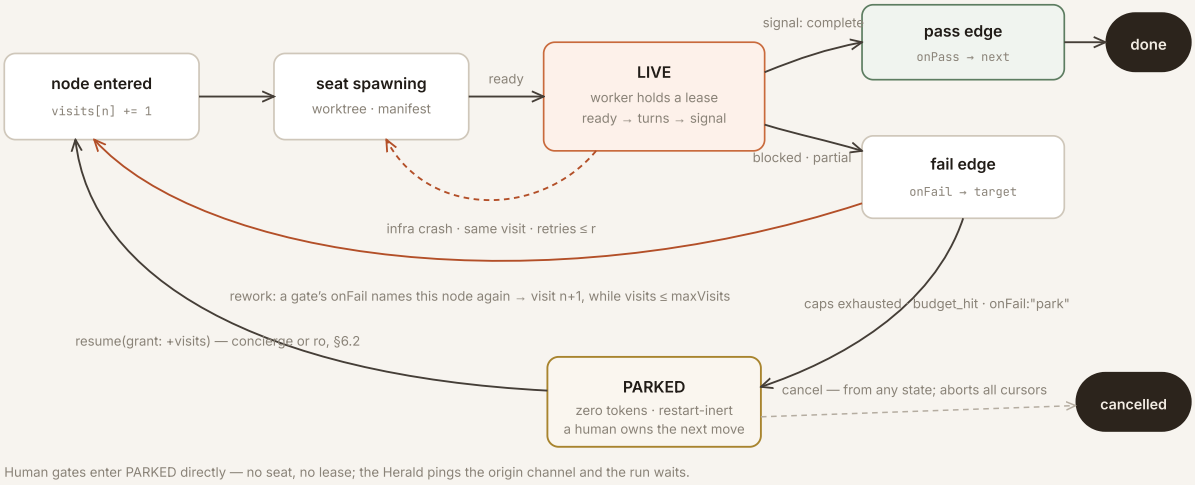


Fig. 5 — one cursor's lifecycle at a worker node. Solid black: the happy path. Dashed rust: same-visit infra retries (crash, auth, rate-limit). Solid rust: the generalised rework loop — re-entry with an incremented visit counter, always bounded by `maxVisits`. Gold: PARKED, the state where humans own the next move; green: the resume grant that re-arms it. Terminal states are dark. Run-level status is the same picture one level up: `running / parked / done / cancelled`.

5.3 How loops terminate — always, by construction

- **Per-node:** entering a node increments `visits[node]`; exceeding `maxVisits` (or a gate's `maxFails`) diverts the edge to **park** with a journal note and a concierge ping — today's "three strikes then leave in `in_review`" for a human" `dispatcher.ts:2756-2783`, generalised and made per-task data.
- **Per-run:** the budget ceilings (spend, wall-clock) are checked before every spawn; a breach parks the run with `budget_hit`. A parked run never burns another token until a human (or the concierge, with authority) resumes it.
- **Statically:** the linter already refused any cycle not cut by a finite counter (§3.3), so the dynamic checks are enforcement, not the proof.

5.4 How parallel branches run and join

A `fanout` node forks the cursor: each arm gets its own worktree and branch — `beckett/task-42.1/arm-2` — via the same `createWorktree` machinery, from the same base SHA (captured exactly as the review base is today `dispatcher.ts:1573-1586`). Arms count against `budget.maxConcurrent` and the global `max_workers` cap; excess arms queue in the existing FIFO. Join strategies:

STRATEGY	SEMANTICS	REUSES
`all-merge`	Wait for every arm's `complete`; merge arm branches into the task branch in arm order; conflict → `onFail`.	`mergeBranchesIntoWorktree()` — already merges dependency branches today `worktree.ts`
`first`	First `complete` wins; engine aborts remaining arms (`handle.abort`), reaps their worktrees.	the cancel path `dispatcher.ts:996`
`quorum`	For verdict-shaped arms (panel review): pass when `quorumK` arms pass; fail when impossible.	done-signal statuses as votes
`judge`	Spawn the judge cast with every arm's diff + summary as its brief; its done-signal names the winning arm (merged) or requests a synthesis pass.	the review-diff builder `dispatcher.ts:1607-1623`

An arm that fails takes the arm's failure into the join: `all-merge` fails fast, `first` ignores it unless all arms fail, `quorum` counts it against, `judge` receives it as context. A cancelled run aborts all cursors — same semantics as today's single worker, mapped over the set. Two failure cases deserve naming, because §7.3 exercises one of them:

- **The silent arm.** An arm whose worker dies without a signal cannot block the join forever: the crash synthesizes a `finished` `base.ts:327-343`, the retry ladder runs, and if retries exhaust, the arm enters the join as failed. Behind all of that, the Sentinel's lease (§6.1) has already alarmed — a join can wait, but never quietly.
- **The half-merged join.** `all-merge` merges arms one at a time and records each merge as its own event; a conflict parks the join with the task branch left at the last clean merge and the conflicting arm named in the park reason. Nothing is force-pushed; recovery is a human (or a synthesis seat) resolving one named conflict.

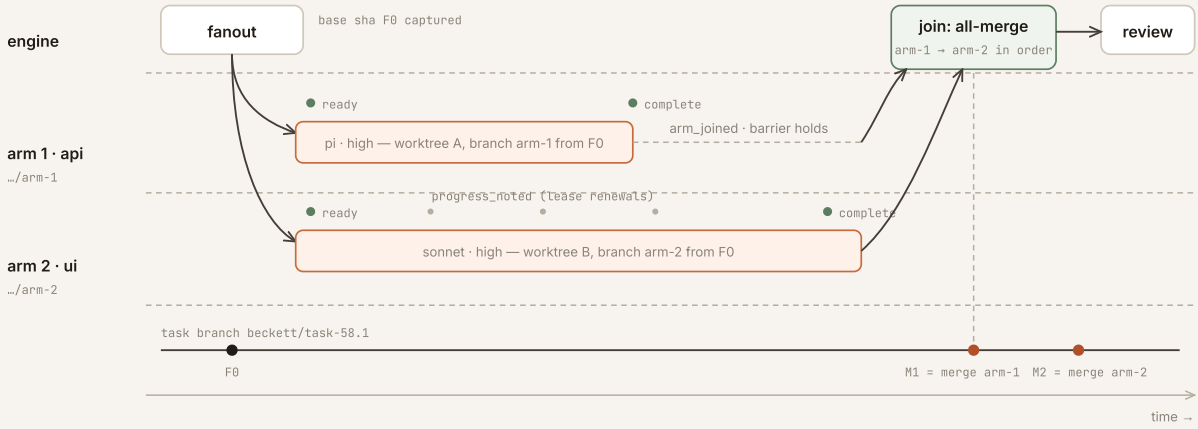


Fig. 6 — fan-out and rejoin on the clock. Both arms fork from one captured base `F0` into isolated worktrees. Arm 1 finishes first and holds at the barrier (`arm_joined`); the join fires only when the slowest arm signals, then merges arm branches onto the task branch in declared order (`M1` , `M2`) before the cursor moves on. Grey dots are lease renewals — the reason a slow arm and a dead arm never look alike (§6.1). §7.2 replays this exact picture with concrete data.

5.5 Steering and observability

Steering stops being “a Plane comment we poll for” and becomes a first-class verb: `beckett task nudge <ref> [--node id] "text"`, called by the concierge when a human replies on the task’s origin channel (the channel linkage already exists — `originChannel` is stamped on every filing `types.ts:114-117`). Un-targeted nudges go to all live cursors; targeted ones to a node’s worker or its buffer. Delivery semantics — receipts, buffering when nothing is live, folding into the next brief, unapplied-nudge drain — are today’s, unchanged `dispatcher.ts:1070-1158`. The journal gains node context per line; `beckett status` renders each run’s DAG with cursor positions, visit counts and spend — which is more visibility than the Plane board offers now, not less.

5.6 Crash recovery

On boot the engine replays each run’s event log (no Plane `prime()` round-trip): PARKED cursors stay parked; LIVE cursors re-staff from their checkpointed worktrees using the existing resumables/session-resume bridge `dispatcher.ts:583-627`. The one-writer-at-a-time discipline the ledger provides today extends per-cursor. This strengthens recovery: today’s restart re-derives intent from Plane state; tomorrow’s replays exactly what the engine was doing.

NEW IN REV 2 · NOTHING FAILS QUIETLY

6

Reliability & observability

Two requirements, verbatim from ro's review: *"when something goes wrong we know about it"* and *"we have clear handshakes the agent can see to make sure the agent understands its environment."* §1.6 showed the gap precisely: today every failure is detected somewhere and announced nowhere, and the worker's model of its own situation is an inference from its `cwd`. This section closes both, with two invariants the rest of the design already leans on: **every abnormal event reaches a human surface within sixty seconds**, and **every worker can prove where it is before it does anything**.

6.1 Failure detection — leases, not vibes

The primitive is a **lease**. Every LIVE seat holds one; it is renewed automatically by any real progress event from the driver stream — turns, tool calls, file changes — the same events the stall watch already counts `base.ts:78, 496-497`. The Sentinel folds renewals into at most one `progress_noted` per seat per minute (telemetry without log spam) and evaluates three clocks against the run's `SuperviseSpec`:

```
interface SuperviseSpec {          // per-run overrides; engine defaults shown. Floors enforced
  leaseS?: number;                  // 90  — silence longer than this = "quiet" (floor 30)
  quietStrikes?: number;            // 2   — quiet windows before the stall alarm (floor 1)
  overrunFactor?: number;           // 1.5 — × the cast's envelope wallClockS → overrun alarm
  checkpointS?: number;             // 300 — WIP checkpoint cadence (OPS-125, now per-run)
  readyS?: number;                  // 60  — spawn → worker_ready deadline (SPAWN_TIMEOUT_MS today)
}
```

- **Quiet → stall.** One missed lease window: the seat is marked `quiet` and gets a status-check nudge — today's strike 1 `dispatcher.ts:1751`, kept. A second window: `alarm_raised(stall)`, and the ladder in §6.2 takes over. The difference from today is not detection — it is that the alarm is an *event in the run log*, so it is folded, rendered in `beckett status`, and announced by the Herald, instead of dying in a logger.warn.
- **Overrun.** The effort envelope stays advisory for the worker, but crossing `overrunFactor × wallClockS` raises `alarm_raised(overrun)` — "this low-effort seat predicted 10 minutes and has been out for 15" — long before the 60-minute hard backstop `config.ts:269` kills anything. The backstop itself survives unchanged, but its firing becomes a *page*, not a log line: by definition it means every softer layer failed.
- **Ready deadline.** A seat that hasn't completed the §6.4 readiness handshake within `readyS` raises `alarm_raised(ready_timeout)` — the spawn-phase hang that today only a 60 s launch timer catches `base.ts:46, 265-275`.

Leases make one distinction structural, and Fig. 6 already drew it: **a slow worker renews; a dead worker doesn't.** No amount of model-side laziness can fake a lease renewal, and no amount of long thinking gets punished for it — the clocks watch silence, not speed.

6.2 The alarm catalogue and the escalation ladder

Alarms are a closed vocabulary, like nodes and events. Each has one automatic first response and one announcement tier; nothing is configurable into silence.

ALARM	RAISED WHEN	AUTOMATIC FIRST RESPONSE	TIER (§6.3)
`stall`	lease expired × `quietStrikes`	abort → checkpoint WIP → retry same visit (today's strike 2 `dispatcher.ts:1771`)	status
`overrun`	1.5× envelope wall-clock	none — informational; repeated → status	status
`ready_timeout`	no `worker_ready` within 60 s	kill, retry launch (existing spawn retry)	status
`refused`	worker's environment check failed (§6.4)	quarantine the seat; re-provision worktree; respawn once	**page**
`silent_exit`	process died, no done-signal	synthesize signal `base.ts:327-343`, retry ladder	status; page on final retry
`hard_cap`	wall-clock backstop fired `base.ts:393-409`	tree killed by driver; checkpoint floor recovers WIP	**page**
`budget_80`	run spend crosses 80% of `budget.usd`	none — forewarning	status
`budget_hit`	a pre-spawn gate refused (§4.3)	park the run	**page**
`join_starved`	join waiting > 2× slowest arm's envelope on a failed/silent arm	fail the arm into the join (§5.4)	status
`mirror_lag`	Phase-1 Plane mirror > 10 min behind	none — execution unaffected by design	trace

The ladder under `stall` / `silent_exit` is deliberately today's ladder — nudge, then abort-checkpoint-retry, then give up `dispatcher.ts:1751-1786, 2150-2236` — with two changes. First, **every rung is an event**, so the operator watches the ladder climb in real time instead of discovering its result. Second, **the top rung pages instead of parking silently**: exhausted retries produce `parked` + a page containing the last checkpoint SHA, the synthesized signal, and the journal tail — everything needed to decide *resume, re-cast, or kill* from a phone.

WHAT "WE KNOW ABOUT IT" MEANS, MADE TESTABLE

For every alarm in the table: inject the fault in a harness test (kill the process, freeze the clock, saturate the budget) and assert that (a) the alarm event is in the run log, (b) the automatic response fired, and (c) a Herald delivery record exists within 60 s. The invariant is property-tested, not promised. A quarterly *fire drill* — one deliberately-wedged canary run — keeps the path honest in production, the same way the outboxes earn trust by being exercised.

6.3 Announcement routing — what lands where, who gets paged

The Herald classifies every run event into one of three tiers. Delivery is durable: pages and status messages go through the same outbox idiom as GitHub publishes `publish-outbox.ts`, so a Discord outage delays an announcement rather than losing it. Status flushes on a 15 s tick; pages flush immediately.

TIER	SURFACE	CARRIES	EXAMPLES
trace	run log + journal only	everything — the full §4.4 vocabulary	`progress_noted`, `edge_taken`, `checkpoint_committed`
status	the task's origin channel (threaded under the filing message)	one-line human sentence + `beckett status` deep link	node transitions, gate verdicts, stall alarms, join resolutions, `budget_80`
page	direct ping to ro (and the concierge's attention queue)	the decision packet: reason, checkpoint SHA, journal tail, the verbs that apply	`parked`, `budget_hit`, `hard_cap`, `refused`, human gates awaiting approval

Three routing rules keep this from becoming noise, because an alarm system that spams gets muted and then it is §1.6 again with extra steps:

- **Status is threaded, not broadcast.** A run's chatter lives in one thread under its own filing message; the channel surface stays one line per run.
- **Pages are decisions, not news.** A page is sent only when a human verb is the next step (`resume`, `grant`, `cancel`, gate approval). News that needs no decision is status, however alarming it sounds.
- **Repeats collapse.** An unacknowledged alarm re-fires into the *same* message (edit, not repost) with an escalating age stamp. One wedged run is one page, aging visibly — not a wall of pings.

Telemetry. The counters ride the same events: `progress_noted` carries turns, tool calls and token deltas from the driver's live telemetry `base.ts:89–92`; `run_done` carries totals; the spend ledger keeps its per-seat `SpendRecord` rows `spend.ts:7–24`. `beckett status` folds all of it per run — DAG, cursor positions, lease states, visit counters, spend against budget — and a nightly one-liner per project (runs finished / parked / spend vs. yesterday) goes out at status tier. That rollup is the whole "dashboard": the design deliberately stops short of a metrics stack until the JSONL outgrows `jq`, which it has not.

6.4 Handshakes — the worker's environment as a contract it can read

Today a worker infers its situation from its `cwd` and the prose of its brief `spawn.ts:320-426`. The fix is a file, not a protocol stack: every spawn writes a **stage manifest** into the worktree beside the done-signal schema — machine-readable, human-readable, and the single answer to "where am I and what am I doing?":

```
// .beckett/stage-manifest.json — written by the Spawn Adapter, read by the worker (and by §7's sims)
{ "run": "#58.1", "node": "ui", "kind": "worker", "arm": 2,
  "visit": { "n": 1, "maxVisits": 3, "retriesLeft": 3 },
  "position": { "upstream":   [ { "node": "design", "status": "passed",
                                  "artifact": "docs/design/spend-page.md",
                                  "summary": "two-panel layout; API shape agreed" } ],
                "downstream": [ "join:merge", "review" ] },
  "workspace": { "worktree": ".beckett/worktrees/58-1-arm2",
                 "branch": "beckett/task-58.1/arm-2",
                 "baseSha": "f31c09a", "scope": ["web/**"] },
  "criteria": [ "spend page renders per-project totals", "loads under 200ms cached" ],
  "budget": { "envelope": { "turnCap": 60, "wallClockS": 2400 },
              "runRemainingUsd": 4.10 },
  "handshake": { "readyBy": "2026-07-12T14:31:00Z",
                 "readyVerb": "beckett seat ready",
                 "refuseVerb": "beckett seat refuse --reason …",
                 "whereami": "beckett whereami" } }
```

Around that file, three handshakes — spawn, progress, completion — each with an acknowledgment the other side can verify:

- **Ready (spawn → work).** The worker's first required act is `beckett seat ready`, which independently checks the observed world against the manifest — current branch equals `workspace.branch`, `HEAD` is `baseSha` or a descendant, the worktree is clean or carries only the expected checkpoint — and appends `worker_ready` with what it saw. On any mismatch the verb exits nonzero, the worker calls `seat refuse` and stops: **a confused agent that halts in ten seconds is a reliability feature; one that guesses is an incident.** Refusal pages, because it means the engine's bookkeeping and the filesystem disagree — the one inconsistency that must never be worked through.
- **Progress (work).** Nothing to remember: leases renew from ordinary activity (§6.1). `beckett whereami` is available the whole time and answers from the manifest *plus* the engine's live fold — node, visit, lease state, budget left, steers pending — so mid-task disorientation (after a compaction, say) has a canonical cure the agent can invoke itself.
- **Done (work → engine).** The done-signal rides the harness's structured output as today `claude.ts:598`. The engine's ack is the fsync'd `signal_received` event (§4.3 step 1); if the daemon dies between a worker's exit and that fsync, boot-time recovery re-derives the signal from the driver transcript — the same resumables bridge crash recovery already uses `dispatcher.ts:583-627`. A worker's work is therefore never both finished and unrecorded.

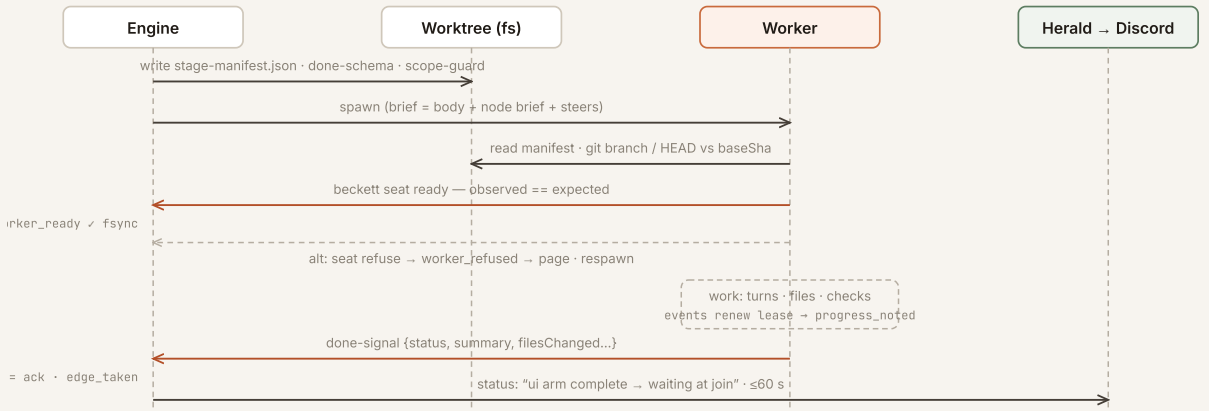


Fig. 7 — the seat handshake, end to end. The manifest is written before the worker exists; the worker proves its environment before it works; the engine's fsync is the completion ack; the Herald tells a human within a minute. The dashed *refuse* alternative is the whole point: disagreement between bookkeeping and filesystem halts in seconds and pages, instead of being guessed through.

THE QUESTION A WORKER ASKS	TODAY'S ANSWER	REV-2 ANSWER
What branch/worktree should I be in?	implied by `cwd`	`workspace.*`, verified at `seat ready`
What stage am I, in what larger shape?	inferred from brief prose	`node`, `kind`, `position.upstream/downstream`
What did the stage before me produce?	sometimes pasted into the brief	`position.upstream[].artifact/summary`, on disk
How much runway do I have?	unknown	`budget.envelope` + `runRemainingUsd`
Which attempt is this?	unknown	`visit.n` of `maxVisits`
Did my finish get recorded?	unknowable	`signal_received` fsync + boot-time re-derivation

6.5 The stability posture, in one paragraph

The posture is: **local, append-only, announced.** Execution state never leaves the machine; every transition is one fsync'd line in one file; every abnormal line reaches a human inside a minute; every process that touches a worktree first proves it belongs there; and every automatic recovery bottoms out, deliberately, at a parked run and a page — a system that knows when to stop is the one you can leave running overnight. Nothing in this section is novel infrastructure; it is the discipline the outboxes and the journal already practise, applied to the one place it was missing: the moment things go wrong.

NEW IN REV 2 · THE ENGINE ON PAPER, BEFORE IT EXISTS IN CODE

7

Three simulated deployments

Three real-shaped tasks, walked through the engine event by event: the boring case, the parallel case, and the case where things break. Timestamps, specs, event logs and manifests are simulated but exact — each line is one the §4.4 vocabulary can actually express, which is the test that the design has no gaps a demo would hide.

7.1 Simulation A — a one-pass backend fix (the everyday ticket)

OPS-208 "journal --tail N returns N−1 lines." The concierge files with no `--flow`; the engine compiles the cast (`pi`, effort `low`) into the built-in one-pass spec and prints it back — the gate that today is silently inferred is now visible at filing:

```
$ beckett task file --project beckett --title "journal --tail off-by-one" --cast implement=pi:low
→ compiled flow: implement(pi:low, retries 3) → done   [no review: effort low → one-pass]
→ run #61 opened · runs/61.jsonl · budget: defaults · supervise: defaults (lease 90s)
```

```
14:02:11 run_opened        {spec: one-pass, origin: #ops-tickets}
14:02:11 node_entered      {node: implement, visit: 1}
14:02:12 seat_spawned      {seat: 61/implement/1, cast: pi:low, branch: beckett/task-61,
                            worktree: 61, baseSha: a41f2c9}          ← manifest written first
14:02:31 worker_ready      {observed: {branch: beckett/task-61, head: a41f2c9}} ← handshake, 19 s in
14:03:30 progress_noted    {turns: 4, files: [src/progress/journal.ts], tokens: 18k}
14:04:30 progress_noted    {turns: 7, files: +[journal.test.ts], tokens: 41k}
14:06:02 signal_received   {status: complete, summary: "off-by-one in tail slice; test added",
                            checksRun: ["bun test journal"]}
14:06:02 edge_taken        {from: implement, to: done, why: pass}
14:06:03 run_done          {outcome: done, seats: 1, spend: $0.14, wallClock: 3m52s}
14:06:09 status → #ops-tickets thread: "#61 done in one pass — journal tail fixed, test added,
                           $0.14. Branch beckett/task-61 queued to publish."
```

Total: one seat, four minutes, six status-tier words of human attention. The point of Simulation A is what it *doesn't* show: no review seat was spawned because the spec said so out loud, not because a model knob implied it; and if this had been rev-1's engine, lines `worker_ready` and both `progress_noted` rollups simply would not exist — the run would have been dark from spawn to signal.

The worker's manifest for seat `61/implement/1`, abridged: `node: implement`, `kind: worker`, `position: {upstream: [], downstream: ["done"]}`, `visit: {n: 1, maxVisits: 3}`, `criteria: ["tail N returns N lines", "regression test"]`, `budget.envelope: {15 turns, 600 s}`. Its first act was `beckett seat ready`; its last was the done-signal. It never needed to know a flow engine existed — the compatibility invariant of §8.4, demonstrated.

7.2 Simulation B — a backend + frontend feature that fans out and rejoins

WEB-58 / run #58.1 "spend report page" — a page over the spend ledger: an API endpoint (backend) and the page itself (frontend), separable after a short design pass. The concierge composes this one deliberately:

```
$ beckett task file --project web --title "spend report page" --channel #web \
    --flow 'design(opus:high) → [api(pi:high) | ui(sonnet:high)] ⇒merge → review(sonnet:high)^2'
→ 5 nodes · fanout isolation: worktree-each · join: all-merge, arm order [api, ui]
→ worst case priced at filing: 1 + 2 + 1 seats × visits ≤ 2 → echo "≤ 7 seats, est ≤ $9"
```

The run, stage by stage — this is Fig. 6 with real values:

```
09:14:02 run_opened        {nodes: 5, budget: {maxConcurrent: 2}}
09:14:02 node_entered      {node: design, visit: 1}
09:14:03 seat_spawned      {seat: 58.1/design/1, cast: opus:high, branch: beckett/task-58.1}
09:14:24 worker_ready      ···
09:26:40 signal_received   {status: complete, summary: "two panels; GET /api/spend?window=30d;
                            artifact docs/design/spend-page.md"}
09:26:40 edge_taken        {from: design, to: fanout, why: pass}
09:26:41 node_entered      {node: fanout}   ← base F0 = f31c09a captured from task branch HEAD
09:26:41 seat_spawned      {seat: 58.1/api/1·arm1, branch: beckett/task-58.1/arm-1, baseSha: f31c09a,
                            scope: ["src/**"]}
09:26:42 seat_spawned      {seat: 58.1/ui/1·arm2,  branch: beckett/task-58.1/arm-2, baseSha: f31c09a,
                            scope: ["web/**"]}      ← both manifests carry design's artifact+summary
09:27:0x worker_ready      ×2   ← each arm verified its OWN branch — the §6.4 check is per-seat
09:41:13 signal_received   {seat: ···arm1, status: complete, checksRun: ["bun test spend-api"]}
09:41:13 arm_joined        {arm: 1, branch: arm-1}          ← barrier holds; ui still live
09:41:19 status: "#58.1 api arm done, waiting on ui (lease healthy, 34k tokens in)"
09:52:47 signal_received   {seat: ···arm2, status: complete}
09:52:47 arm_joined        {arm: 2}
09:52:49 join_resolved     {strategy: all-merge, merged: [arm-1 → M1 41bc2e0, arm-2 → M2 5580f1d],
                            conflicts: none}
09:52:50 node_entered      {node: review, visit: 1}         ← reviewer's manifest: upstream =
                                                             design + BOTH arms, with summaries
10:01:08 gate_decided      {verdict: pass, rubric: criteria-vs-diff}
10:01:08 run_done          {seats: 4, spend: $6.80, wallClock: 47m}
```

What Simulation B establishes, concretely:

- **The arms never raced.** Isolation was worktree-each; the `ui` arm's scope-guard (`web/**`) would have denied a stray write into `src/` at the tool layer, exactly as the guard works today `scope-guard.ts:180` — parallelism adds worktrees, not new trust assumptions.
- **The join is ordinary git.** `M1`, `M2` are merge commits on the task branch made by the same machinery that merges dependency branches today `worktree.ts`; a conflict would have parked the join with arm-2 named, task branch resting at `M1` (§5.4).
- **One reviewer saw the whole.** Its manifest's `upstream` carried the design artifact and both arm summaries — the fan-out is invisible in the diff but legible in the contract. Under today's engine this task is either one serial mega-ticket or two hand-filed siblings with a human doing the join in their head.

7.3 Simulation C — a failure, detected, recovered, and paid for honestly

OPS-233 "durable retry backoff in publish-outbox" — a `reviewed` preset (implement pi:high → review sonnet:high, `maxVisits 3`), filed at 16:40. Everything goes wrong that quietly goes wrong today; watch what surfaces this time:

```
16:40:55 run_opened · node_entered{implement,1} · seat_spawned · worker_ready   ← nominal start
16:52:xx progress_noted    ··· steady ··· then nothing after 16:58:07     ← harness wedged mid-turn
16:59:40 lease quiet      {seat: 233/implement/1, idle: 93s} → status-check nudge   ← strike 1
17:01:12 alarm_raised      {type: stall, idle: 185s, strikes: 2}
17:01:12 status: "#233 implement seat stalled (3 min silent); aborting to checkpoint and retrying"
17:01:13 checkpoint floor  = c99d47e (17:00:03, OPS-125 cadence)   ← at most 60 s of work lost
17:01:14 signal_received   {status: synthesized, subtype: error_abort_stall}    ← never silent
17:01:15 edge_taken        {implement → implement, why: retry, visit: 1, retriesLeft: 2}
17:01:16 seat_spawned      {seat: 233/implement/1·r2, resume: from c99d47e}
17:14:59 signal_received   {status: complete}  → edge_taken {implement → review}
17:22:30 gate_decided      {verdict: fail, reasons: ["backoff caps at 2 retries, spec says 3",
                            "no jitter — thundering-herd note in criteria"]}
17:22:30 edge_taken        {review → implement, why: fail, visit: 2 of 3}   ← rework, bounded
17:22:31 seat_spawned      {···/implement/2}   ← its manifest: visit.n=2, upstream review verdict
                                                 inline — the worker KNOWS why it's back
17:36:44 signal_received   {status: complete} → review(2) → 17:44:03 gate_decided {pass}
17:44:04 run_done          {outcome: done, seats: 5, spend: $4.90, wallClock: 63m,
                            alarms: [stall×1], retries: 1, reworks: 1}
```

And the part that answers §1.6 directly — **what ro's Discord actually showed**, in the #ops-tickets thread for #233, unprompted:

```
beckett  16:41  filed #233 — implement(pi:high) → review(sonnet:high), ≤3 visits, est ≤ $6
beckett  17:01  ⚠ implement seat went silent for 3 min — aborted to checkpoint c99d47e,
                retrying (1 of 3). Nothing lost past 17:00.
beckett  17:22  review sent it back: backoff cap + missing jitter. Visit 2 of 3.
beckett  17:44  ✓ #233 done — 1 stall recovered, 1 rework, $4.90, 63 min. Publishing.
```

Four lines, zero questions asked. Under the current system the same afternoon reads: worker goes quiet at 16:58, the stall nudge and abort happen inside log files `dispatcher.ts:1751-1786`, and the first human signal is ro noticing at 18:30 that #233 is still `in_progress` and asking the concierge what happened. The failure and the recovery are identical in both worlds — **the difference is purely who got told, and when.**

Had the second retry also stalled, or the third visit failed review, the run would have **parked and paged** with the decision packet (§6.3): last checkpoint SHA, both review verdicts, spend so far, and the applicable verbs — `resume --grant visits=1`, `cancel`, or re-cast the seat. The ladder has a top, the top has a human, and the human has everything needed to rule in one read.

WHAT THE THREE SIMULATIONS PROVE TOGETHER

Every line above is expressible in the nineteen-event vocabulary of §4.4 — no simulation needed an event the model lacks, which is the paper-stage equivalent of a passing integration test. A: defaults stay invisible and cheap. B: parallelism is worktrees + ordinary merges, priced at filing. C: failure is loud, bounded, and recoverable from a checkpoint that is never more than one cadence old. These three transcripts are the acceptance tests the Phase-0 shadow build (§9.1) should replay verbatim.

AUTHORING · WHAT THE CONCIERGE WRITES

8

The authoring surface

Filing keeps its shape: `beckett task file` with title, body, criteria, project, channel. The flow arrives three ways, cheapest first — and the cheapest way is not writing one.

8.1 Defaults — most filings never mention a flow

No `--flow` → the engine compiles the spec from the cast exactly as today: effort `low|medium` → one-pass; `high|xhigh` → implement + review gate; `reviewTier` respected. Zero behaviour change, zero new typing for the everyday ticket. (One deliberate improvement: the compiled spec is printed back on filing — the gate becomes visible instead of inferred silently. Simulation A shows the echo.)

8.2 Flow presets — named shapes in `~/.beckett/flows.json`

The OPS-110 pattern, re-applied: hot-reloaded, user-editable, validated on load, seeded with the vocabulary we already know we want:

PRESET	SHAPE	REPLACES / ENABLES
`one-pass`	implement → done	today's *self* tier, said out loud
`reviewed`	implement → review ⟳≤3 → done	today's *fresh* tier
`intensive`	design → human gate ⟳≤2 → implement → review → done	the whole INT board, as 4 nodes of data
`bake-off`	fanout(n candidates) → judge → review → done	Fig. 3 — impossible today
`panel-review`	implement → fanout(correctness, security, taste) → quorum 2/3 ⟳ → done	parallel reviewers — impossible today
`iterate-until`	implement ⟳≤N against a model-gate rubric (eval/tests) → done	per-task iteration budgets — the constants, freed

Invocation: `--flow-preset bake-off`, with per-node overrides via `--flow-set judge.cast.model=claude-fable-5` or a partial-JSON merge — same precedence story as cast presets (explicit beats preset, per stage) `presets.ts:119-142`.

8.3 A shorthand for composing fresh — the one-liner grammar

For shapes worth composing on the spot, a tiny expression syntax compiled to a `FlowSpec` (and echoed back as the expanded JSON before filing):

FORM	MEANING	EXAMPLE
`stage(cast)`	worker node	`design(opus:high)`
`?human` / `?cast`	gate	`?human` · `?haiku:low(rubric=criteria)`
`a → b`	sequence	`implement(pi:med) → review(sonnet:high)`
`[a | b | c]`	fanout arms	`[impl(pi:high) | impl(claude:high) | impl(codex:high)]`
`⇒judge(cast)`	join	`⇒judge(fable:high)` · `⇒merge` · `⇒first` · `⇒quorum:2`
`^N`	visit cap	`review(sonnet)^2` — two rework rounds, then park

```
# what I'd actually type when ro asks for a bake-off with a design gate:
beckett task file --title "…" --project foo --channel <id> \
  --flow 'design(opus:high) → ?human → [impl(pi:high)|impl(claude:high)]
          ⇒judge(fable:high) → review(sonnet:high)^2'
```

The grammar is sugar only — it must compile to the §3.2 model and validate identically. If it ever wants a feature the model lacks, the model wins and the feature waits for a design doc.

8.4 What stays exactly as it is

- **`beckett plan` and the task tree.** Inter-task DAGs (`--needs`, `#N.x`, `blockedBy` promotion) are the right tool for separate deliverables and keep working unchanged; a plan node may now also carry a `flow`. Flows are intra-task; plans are inter-task. The two compose and do not compete.
- **Casting doctrine.** Presets, the roster, effort envelopes, Fable confirm-first — all untouched. A flow references seats; the cast doctrine still decides who may sit in them.
- **The worker's world.** Brief in, worktree, scope-guard, steering nudges, done-signal out — now with one addition it may freely ignore: the stage manifest (§6.4). A worker cannot tell whether it is running under the old dispatcher or the stage manager — that is the compatibility invariant the whole migration hangs on, and Simulation A exercised it.

GETTING THERE · THREE REVERSIBLE PHASES

9

Migration off Plane

Plane provides four things (§1's observation): issue storage, a state column, comments-as-transport, and a board humans can look at. Each gets an explicit replacement, and each phase is independently shippable and reversible.

PLANE CONCEPT	REPLACEMENT	NOTES
Issue (title/body/criteria)	Task record in `~/.beckett/tasks.json`	already exists and already mirrors every ticket `store.ts`
Workflow state column	`FlowRun` cursor + status (§5.1)	strictly more expressive; branch-status mapping `store.ts:141–152` becomes a projection of the run
Description fence blocks (`beckett-cast/-deps/-project/-branch/-start-state`)	Typed fields on the task record	ends the Markdown smuggling; `parseCast` survives only as an importer
Comments (steering)	`beckett task nudge` + Discord origin-channel replies	transport was always Discord → concierge → Plane → poller; this deletes the middle two hops and the ~5 s latency
Comments (audit trail)	Run event log + journal	both append-only, both local, journal already richer `journal.ts`
Ticket IDs (`OPS-42`)	Local per-prefix sequence	`nextTaskNumber` already allocates `#N` `store.ts`; keep the OPS-N surface so journals, history and habits survive
`blockedBy` dependencies	`needs` in the task tree	already dual-recorded today; the local copy becomes primary
Board UI	`beckett status` DAG view; optional read-only web board later	the honest loss — see §10.2

Fig. 8 — migration phases. Each phase ships alone, proves itself in production, and can be rolled back before the next begins.

9.1 Phase 0 — shadow (engine exists, Plane rules)

Build the interpreter + run store + linter. Compile every incoming ticket's cast into its equivalent built-in spec (§8.1) and run the engine *as a shadow*: it receives the same events, folds its own run record, and logs any divergence between the edge it would take and the transition the dispatcher actually performs. The dispatcher's switch keeps driving. This phase has no behaviour change at all and produces the evidence for the flip: N days of zero-divergence on live traffic — the same shadow-then-flip discipline used for the ambient classifier. The Sentinel and Herald ship here too, watching *today's* workers: the §6 alarm path earns trust on the old engine before the new one depends on it, and §7's three transcripts become the shadow's replay fixtures.

9.2 Phase 1 — flip (engine rules, Plane mirrors)

The run store becomes authoritative. The dispatcher's `onStateChanged` shrinks to an adapter: engine edges call the spawn/steer/park/publish primitives directly; a projection maps each run state onto the closest legacy Plane column and writes it through the existing `advance-outbox` — Plane can lag, break or 404 without touching execution (the outbox already assumes it will `advance-outbox.ts`). Human state-flips on the Plane UI stop being inputs; the sanctioned control surface is the concierge's verbs (`task nudge / pause / resume / cancel / courier`), which is where ro actually operates from anyway — Discord. Custom flows (`--flow`, presets, the grammar) unlock in this phase. Rollback is a config flag: point the poller back at Plane and re-arm the switch.

9.3 Phase 2 — retire

After the mirror has run quiet for a few weeks: export every board to archive JSONL (issues + comments, for the historical record), delete `src/plane/client.ts` / `poll.ts` and the mirror projection, keep `cast.ts` parsing as a legacy importer, and decommission the Plane instance. The `OPS-` prefix lives on as a local sequence so identifiers stay continuous.

WHAT DELIBERATELY DOES NOT CHANGE, EVER

Worktree and branch conventions (`beckett/task-N.x`), the scope-guard, done-signal schema, effort envelopes, cast presets and the Fable handshake, the journal and its CLI, the publish-outbox and courier handoff, `beckett plan` semantics, and the `#N` task tree. The blast radius is confined to: who decides the next stage (engine vs. Plane-state switch), where that decision is stored (local run log vs. remote column) — and, new in rev 2, who is told when a decision goes wrong (the Herald vs. nobody).

HONESTY · COSTS AND UNKNOWNS

10

Tradeoffs, risks, open questions

10.1 We are building a workflow engine — the classic trap

Half-built workflow engines are where codebases go to die: they grow expression languages, conditionals, sub-flows, and eventually a worse programming language nobody can debug. The defence here is structural, not aspirational: four node kinds, closed set; edges carry no predicates beyond pass/fail; no data flows through the graph except briefs and diffs; the linter refuses anything unbounded; and the bar for a fifth node kind is a design doc like this one. Rev 2 extends the same fence to the new surfaces: nineteen events, ten alarms, three announcement tiers — all closed vocabularies. If any algebra ever feels cramped, that pressure should produce one new member, not an expression language.

10.2 We lose the board

Plane's genuine value is a glanceable, drag-a-card UI for a human. `beckett status` plus Discord updates cover the operational loop we actually use (ro files, steers and approves from Discord today; the board is mostly read). But "mostly" is doing work in that sentence: if the visual board matters, the mitigation is a small read-only web view over the run store — a render of the same JSONL, no write path, no schema pressure — shipped only if its absence is actually felt. This is the one place the proposal subtracts something real; it should be called out to ro as such, not papered over.

10.3 Fan-out × iteration = cost explosion

A 3-arm bake-off at implement-high is roughly 3× an implement ticket before the judge is paid. Guards: per-run `budget.usd` (checked pre-spawn, §5.3) with the `budget_80` forewarning and the `budget_hit` page (§6.2); fanout arms count against the concurrency caps; Fable-in-any-seat keeps its confirm handshake; and the concierge's filing echo prints the worst-case seat count (`nodes × maxVisits × arms`) so the expensive shape is priced before it runs, not discovered on the spend report — Simulation B's echo shows the format. Culturally: presets encode the cheap defaults; exotic flows are typed deliberately.

10.4 Correctness burdens move in-house

Today Plane's column is a crude but externally-visible source of truth; tomorrow a local event log is. Join deadlocks (an arm that never signals), park/resume races, replay determinism, and cursor/ledger consistency become our bugs. Mitigations: the event log is append-only and replay is pure (the same property that makes the outboxes trustworthy); the hard wall-clock backstop `config.ts:269` and the Sentinel's lease (§6.1) together bound every worker, so a silent arm cannot block a join forever — and cannot block it *quietly* at all; and the join/park state machine is small enough for exhaustive property tests. This is real engineering cost — it is the price of owning the semantics, and it is the same class of machinery (ledgers, outboxes, checkpoint replay) the team has already built well three times.

10.5 Two sources of truth during Phase 1

The mirror is strictly one-directional by design; the moment a human edit on Plane can change execution, we have split brain. Enforced by code (mirror writes only) and by doctrine (the Plane UI is read-only from the flip onward). The rollback flag keeps the exit open until Phase 2 deletes the question.

10.6 Alarms can cry wolf NEW IN REV 2

The failure mode of §6 is not silence but noise: a lease window tuned too tight turns long legitimate thinking into stall pages, and a paged human who learns to ignore pages is worse off than one who was never paged. Defences: the clocks watch *silence, not speed* (a thinking harness still emits turn events); thresholds are per-run data with sane defaults, so one chatty project can't be fixed by detuning the fleet; repeats collapse into one aging message (§6.3); and the quarterly fire drill doubles as a false-positive audit — if the drill is the only page that month, the thresholds are right. The metric that matters is the one §1.6 implies: *time from abnormal event to human awareness*, and it should be minutes, boring, and flat.

10.7 The manifest can drift from reality

A contract file is only as honest as its writer; a bug in the Spawn Adapter could describe a world that isn't there. This is why the readiness handshake verifies *independently* — `seat ready` asks git, not the manifest, what branch and HEAD are — and why refusal pages rather than retrying forever: manifest-vs-filesystem disagreement is a bookkeeping bug by definition, and the design's posture is to halt and surface those, never to reconcile them silently.

10.8 Open questions for ro

1. **Board or no board?** Is a read-only web view a launch requirement for Phase 1, a later nice-to-have, or unnecessary? (My read: later; Discord + `beckett status` is where the loop actually lives.)
2. **Concierge authority at parks.** When a run parks on cap-exhaustion, may the concierge grant one extra visit on its own judgment, or is a parked run always a human decision? (I lean: human, always — parks are rare and are exactly the moments you'd want to see.)
3. **Grammar vs. JSON as the primary authoring surface.** The one-liner (§8.3) is expressive but is a new mini-language to maintain; presets + JSON may be enough. Ship the grammar in v1 or hold it?
4. **Should `beckett plan` fold in eventually?** An inter-task DAG is a flow whose worker nodes file sub-tasks. Unifying is elegant but risks the tarpit (§10.1). I propose: not in v1, revisit once flows have months of production shape.
5. **Per-arm steering surface.** `--node` targeting (§5.5) is powerful but chatty in Discord. Is "nudge the whole run" enough for v1?
6. **History.** Is the Phase 2 JSONL export of old Plane boards sufficient, or do you want OPS/INT history queryable (e.g. imported as read-only run records)?
7. **Paging boundaries.** NEW §6.3 pages on parks, budget hits, hard caps, refusals and human gates. Too much, too little? And should quiet hours exist — a window where pages queue and only a wedged *everything* breaks through?

RECOMMENDATION

Build it in the phase order given, and hold the line on smallness: four node kinds, six seed presets, nineteen events, three announcement tiers, no expression language, board UI deferred. Phase 0 is almost pure addition and produces its own go/no-go evidence twice over — the zero-divergence log for the interpreter, and a quarter of Sentinel/Herald service on the *old* engine before anything depends on them; the flip is one config flag with a rollback; Plane dies only after it has been demonstrably decorative for weeks. The payoff is the exact capability requested across both reviews — stages, iterations and parallelism dictated per task by the concierge at filing time, on an engine that announces its own failures and hands every worker a contract it can verify — bought with machinery Beckett already trusts: JSONL ledgers, worktrees, casts, and done-signals.

APPENDIX A · BEHAVIOURAL EQUIVALENCE

Today's lifecycles as flow specs

These three specs are the compatibility contract: at Phase 0/1 cutover, every filing compiles to one of them, and the engine must reproduce the current dispatcher's decisions on each — same spawns, same caps, same parks. The constants shown are today's hardcoded values (§1.3), now merely defaults.

A.1 — one-pass (today: effort low/medium, tier "self")

```
{ "version": 1,
  "entry": "implement",
  "nodes": {
    "implement": {
      "kind": "worker",
      "cast": { "harness": "pi",
                "effort": "medium" },
      "onPass": "done",
      "onFail": "park",
      "retries": 3
        // = MAX_IMPLEMENT_RETRIES
    }
} }
```

The zero-diff withhold (self-tier finish with no changes → route to review instead `dispatcher.ts:2283-2293`) becomes a linter-inserted model gate on this spec — the special case turns into visible structure.

A.2 — reviewed (today: effort high/xhigh, tier "fresh")

```
{ "version": 1,
  "entry": "implement",
  "nodes": {
    "implement": {
      "kind": "worker",
      "cast": { "harness": "pi",
                "effort": "high" },
      "onPass": "review",
      "onFail": "park",
      "maxVisits": 3,   // = MAX_REWORK_CYCLES
      "retries": 3
    },
    "review": {
      "kind": "gate",
      "by": {
        "cast": { "harness": "claude",
                  "model": "claude-sonnet-5",
                  "effort": "high" },
        "rubric": "criteria-vs-diff"
      },
      "onPass": "done",
      "onFail": "implement",
      "maxFails": 3
    }
} }
```

A.3 — intensive (today: the entire INT board — two enum values, five modules, one Plane project)

```
{ "version": 1, "entry": "design", "nodes": {
  "design": {
    "kind": "worker",
    "cast": { "harness": "claude", "model": "claude-opus-4-8", "effort": "high" },
    "artifact": "docs/design/<id>.md",
    "onPass": "completeness", "onFail": "park", "maxVisits": 2
  },
  "completeness": {
    "kind": "gate",
    "by": { "cast": { "harness": "claude", "model": "claude-haiku-4-5", "effort": "low" },
            "rubric": "design-doc completeness" },
    "onPass": "approve", "onFail": "design", "maxFails": 2   // = MAX_DESIGN_CYCLES
  },
  "approve": {
    "kind": "gate", "by": "human",                            // PARKED: zero tokens
    "onPass": "implement", "onFail": "design", "maxFails": 3
  },
  "implement": {
    "kind": "worker", "cast": { "harness": "pi", "effort": "medium" },
    "onPass": "review", "onFail": "park", "maxVisits": 3
  },
  "review": {
    "kind": "gate",
    "by": { "cast": { "harness": "claude", "model": "claude-sonnet-5", "effort": "high" },
            "rubric": "criteria-vs-diff" },
    "onPass": "done", "onFail": "implement", "maxFails": 3
  }
} }
```

SOURCES & VERIFICATION

Rev-1 ground truth verified against the working tree at commit `6fac13d` (2026-07-12); rev-2 additions (§1.6, §4, §6) verified against the same lineage at `29e4de0`. Primarily:

`src/plane/types.ts, cast.ts, client.ts, poll.ts, presets.ts · src/dispatch/dispatcher.ts, spawn.ts, advance-outbox.ts, publish-outbox.ts · sr`

Authored by Beckett's worker seat (Claude Fable 5, high thinking): rev 1 for OPS-151, rev 2 for OPS-152. Design proposal only — no code changes accompany this document. Build source: `docs/design/run-of-show/` (HTML + inline SVG → headless Chrome).